Erik Eismann, Marc Stamminger *Erlangen, 13.10.2026*

Compute Rasterization And Tracing For 3D Rendering (exercise sheet 1)

Welcome to the first exercise of CRAT3R - Compute Rasterization And Tracing For 3D Rendering. In this exercise, you will learn how to implement a simple GPU particle system using compute shaders. To achieve this, you will discover how to create buffers and images in Vulkan, bind them to shaders using descriptors and create and dispatch a compute pipeline.

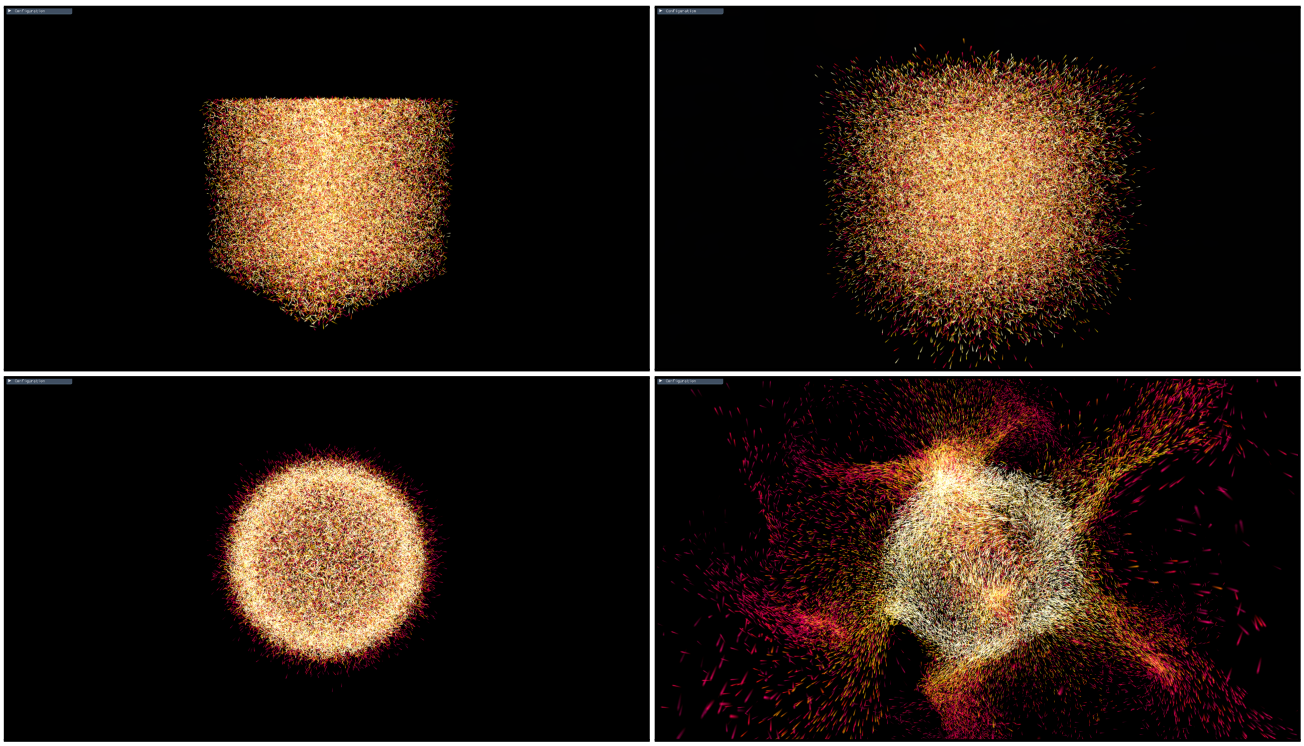


Figure 1: Rendered results of this exercise as you progress through the assignments.

Some important tips for the assignments:

- **Plan a significant amount of time for the assignments**. Start early, plan ahead and decide what to do and when. For the first exercise, aim to complete at least three sub-assignments per week.
- **Beware the validation layers!** They are your friend, telling you when your implementation doesn't make sense. Always fix all warnings from the validation layers before moving on to the next task. Note: they are only enabled by default when compiling in Debug mode.
- **Prepare well for your submission.** You are expected to present your results and highlight what you have learned. If you can explain where things went wrong and how you resolved the issues, you are on the right track. It is not enough to simply complete the tasks; you should also be able to explain the reasons behind the code you have written.
- **Version control.** Use Git or something similar and commit your changes after every task.

Assignment 1.0 Initial Setup

First, download and extract the code from StudOn. To build the project, you will need a compiler with C++20 support, CMake, as well as the Vulkan SDK installed on your system. For Windows download the SDK from `https://vulkan.lunarg.com/`. For Linux install the relevant packages (e.g. on Ubuntu: `libvulkan-dev`, `vulkan-utility-libraries-dev`, `vulkan-validationlayers`, and optionally `vulkan-tools`). The `glslc` shader compiler is also required. If you download the full SDK from the website, it should come packaged with it. Otherwise, you will need to install it separately. Then, ensure that it is on your shell's search path.

To build the project, first run `cmake -S . -B build -G Ninja` on Linux, or on Windows configure the project using the CMake GUI. Then, either run `cmake --build build` or use Visual Studio to compile. If there are no errors, you should then find the `a01` executable in the `build` directory. Running this should open a window with a green background and a small UI on top.

Next, take a look at the project layout. All your implementation will be done in the files `main.cpp` and `particles.comp`. Much of the initialization work for Vulkan has already been done for you. The crat3r core library initiates Vulkan, selects and creates a device, and creates a window, surface and swapchain. A particle renderer is also provided for the first assignment. You don't need to understand what it does for now; it's just there to help you see something on your screen right after you add the necessary storage buffer.

Assignment 1.1 Storage Buffer Creation

First, we need a storage buffer to hold the particles on the GPU. Refer to `particle.h` to see what data is required for each particle. Then, start your implementation of `main.cpp` by allocating a storage buffer and filling it with some initial data. As we will be creating multiple buffers throughout this assignment, the first step is to implement the `createBuffer` function, which creates a buffer and allocates memory for it. Next, use this function in `createStorageBuffer` to create the storage buffer, and then map the memory to upload data to it. If you do this correctly, you should see a cube of particles similar to the top-left image of Figure 1.

A note on performance: Allocating a new block of memory for each buffer is very inefficient. Additionally, the minimum size and maximum number of allocations are typically restricted. However, as we will only be allocating a few buffers in this assignment, we will stay well below these limits. Therefore, we will not yet use a dedicated memory allocator in this assignment, but stick with `vkAllocateMemory`.

Assignment 1.2 Staging Buffer

While the first approach of creating a host-visible storage buffer is practical, it is rather inefficient. Since we will never access the buffer from the CPU again after the initial upload, it would be better to allocate it in device-local memory. However, it is not possible to upload data to device-local memory directly. A common pattern instead is to upload the data to a host-visible staging buffer and then copy it to a device-local buffer.

You are now asked to implement this in `createStorageBuffer`. First, store the handles to the current host-visible buffer locally and update its usage flags. Then, create a second buffer in device-local memory and copy the data from the first staging buffer to it. Finally, free the staging buffer. If your implementation is correct, you should see the exact same result again, but rendered more efficiently.

Assignment 1.3 Storage Buffer Descriptors

We can now see a bunch of efficiently rendered particles, but they are not doing anything interesting yet. Thus, we want to run a compute shader on the particles to move them around. However, we cannot pass buffers directly to shaders in Vulkan, instead, we must create descriptors that point to the buffer. To do this, we need to create a descriptor pool to allocate descriptor sets from, a descriptor set layout to inform Vulkan what the sets look like, and the descriptor sets themselves to point to the storage buffer (and other things later on).

Note that our framework supports mulitple 'frames in flight' to be pre-recorded by the CPU before the GPU actually renders them. This is not an issue for the particle storage buffer, as the frames are still processed by the GPU in order. However, later on we will add uniform buffers that are modified by the CPU for every frame. These require a separate copy for every frame in flight, because the data of the previous frame would otherwise be overwritten by the next one before it has been rendered. For this reason, we will allocate a seperate descriptor set for each frame in flight.

Now, implement the three functions: `createDescriptorPool`, `createDescriptorSetLayout`, and `createDescriptorSets`.

Assignment 1.4 Compute Pipeline

The time has finally come to create the compute pipeline and dispatch the shader. To do this, you need to implement both `createPipelineLayout` and `createPipeline`. Then, dispatch the pipeline on every frame in `recordCommandBuffer`. Initially, nothing will happen because the shader is not yet doing anything. Take a look at `particles.comp`. Modify the position of particle in some way, for example by adding their velocity. You should then see a result similar to the top-right image in Figure 1.

A quick note on shader compilation: All shader files in the `shaders` subdirectory are automatically compiled when `cmake --build` is run, where the resulting SPIR-V code is placed next to them. Hot-reloading is not implemented, so you will have to recompile the project whenever you change a shader.

Assignment 1.5 Uniform Buffers

Although our compute shader can now work on the particles, it cannot yet be controlled from the CPU. To resolve this, we will now add uniform buffers. These will store read-only data that controls the behaviour of the particles and is shared by all shader invocations. As previously mentioned, we will need a separate buffer for each in-flight frame. It is fine to keep uniform buffers in host-visible memory, as they are updated frequently. Additionally, since mapping GPU memory is rather expensive, we will map the buffers once and keep the mapped pointers for the rest of the program.

First, implement `createUniformBuffers`. Next, update `createDescriptorPool` to allocate enough space for the uniform buffer descriptors. Then, add the second binding to the descriptor set layout in `createDescriptorSetLayout`, before finally writing the uniform buffers to the descriptor sets in `createDescriptorSets`. It is at this point that the descriptor sets of the frames differ, as each one references a different uniform buffer.

Now, the data must be copied from the CPU memory to the GPU memory before the frame is rendered. In `updateUniformBuffer`, copy the uniform data to the mapped memory of the current frame's buffer. Finally, uncomment the uniform buffer binding in `particles.comp`. You can now correctly integrate the particle velocies by using the delta time given by the uniform buffer.

Assignment 1.6 Particle Lifecycle

We can now use the uniform data to reduce the life of particles and respawn them if they die. Follow the instructions in `particles.comp`. You should then see a ball of particles similar to that shown in the bottom-left of Figure 1. Feel free to experiment with different forces like gravity and play with the sliders to see how it affects the particles.

Assignment 1.7 Force Field Image

Our particles can now move around, die and respawn. However, their movement is not particularly interesting. To address this, we will add a force field to push the particles around. This field is stored as a three-dimensional image, which the compute shader will then sample.

Start by implementing `createImage` analogously to `createBuffer`. Then use this function to create an image in `createForceFieldImage`. It is not possible to upload data to images directly, so we will have to use a staging buffer again. Note that the image's layout must be transitioned before and after copying the data from the buffer to the image. You can use the `transitionImageLayout` utility function for this. Next, create an image view in `createForceFieldImageView` and a sampler in `createForceFieldImageSampler`.

Then, in a similar way to when adding the uniform buffer, allocate space in the descriptor pool, update the pipeline layout, and write the image view and sampler to the descriptor sets. Finally, uncomment the image sampler binding in the compute shader.

Assignment 1.8 Force Field Effects

As a final step, apply the force field in the compute shader. Follow the instructions to sample the image, apply a force to the particles, and additionally add a drag force. If you have implemented everything correctly, your results should resemble the bottom-right image of Figure 1.